User Roles: Actions Tab & Permission Enforcement

Status report | User Roles > Roles tab > edit any role

The user-friendly solution

The role editor now has two tabs instead of one long scroll (stacked, the dialog was ~1,400 px and the Actions half was never reached).

Tab	Answers
Screens & Modules	What may they **open** (incl. the new Quality checkbox)
Actions (new)	What may they **do**

The design decision that matters most

A role with no grants is not a role that may do nothing: most already hold actions by default. An untouched checkbox would be a lie, so every row states the **effective answer and its source**:

Approve a glass PO step `glass-purchase.approve`	**Allowed, by default for this role** (ticked, LOCKED)
Create a dispatch `dispatch.create`	**Not allowed.** Default: ADMIN, DISPATCH, PRODUCTION (tickable)

- **Green "by default"** rows are ticked and disabled: granting is redundant, and unticking cannot withdraw a default, so an editable box would promise something the backend won't do.
- **Blue "granted here"** = an explicit grant you added.
- **Grey** shows exactly which roles hold the action by default, so you know what to grant.
- **ADMIN** shows a green banner: "Administrator, full access. ADMIN bypasses ordinary permission checks" (not "0 of 20 modules").
- **Search** covers action, screen and role (dispatch, STORE, approve).
- **Footer:** taking away a default requires a code change, a reviewed change because it affects every company. A screen that could withdraw a default would let one mis-click stop the plant invoicing.

Verified end-to-end: grants are real

`operation-grants-e2e.ts`: **28/28** in a sandbox role that is created and hard-deleted. Flow: grant, persist, resolve, enforce, revoke, stop. Also confirmed: junk keys dropped, case/whitespace normalised, de-duplicated; only the granted action is granted (not 21); a grant can only add, never withdraw a default; a deactivated role grants nothing (fail closed).

`Full vitest`	**78 files, 2,210 passing, 0 failures**	`module-allowlist.ts`	**22/22**
`tsc --noEmit`	**clean**	`permission-operations.ts`	**27/27 (12 roles x 14 gates, 54 users, 0 differences)**
`npm run build`	**exit 0, 132/132 pages, /user-roles 19.7 kB**	`operation-grants-e2e.ts`	**28/28**
`permissions-version.ts`	**29/29**	`HTTP dispatch / glass`	**41/41 | 21/21**

Data intact: users 57, role_permissions 7, custom_roles 4, assignments 57, glass POs 79, GRNs 186, invoices 1,552, permissionsVersion 1. All 9 out-of-scope pre-existing changes byte-identical to the session-start backup.

Two corrections owed

- **Build broken, then fixed.** The module-coverage guard used a Map-iterator spread this tsconfig rejects (*npm run build: Failed to compile*). Fixed with `Array.from()`; re-verified exit 0.
- **Two stale assertions** in my own verify scripts asserted the migrations were *not* applied. Now that they are approved, both assert the approved state: the grants column is applied and empty, which is the real safety claim.

Honest status: not everything is done

Step	Status	Detail
4a Propagation	**COMPLETE**	Verified end-to-end
4b QUALITY	**COMPLETE**	Includes the missing checkbox you surfaced
4c Registry	**BATCH 1 OF ~9**	94 to 80 route files still carry a hardcoded role gate
4d Action enforcement	**PARTIAL**	The 21 operations are enforced; ~280 withHandler routes not yet action-gated
4e UI	**COMPLETE**	Built and usable for the 21 wired operations
4f Branch scopes	**NOT STARTED**	READ_ONLY / TRANSACT_ONLY still unenforced

The system is now usable and permanent: actions can be granted and revoked from the UI and the backend enforces them. What remains is coverage: 80 more route files must come under the registry before every act in the ERP appears in the Actions tab. Currently listed: dispatch (3), glass purchase (7), work orders (6), FG sale (2), tax invoices (3).

Recommended next step: 4c batch 2 (~10 more files per round). Continue, or try the Actions tab first and say what to change?